

Malware Threats

BCS2420

Dr. Ashish Sai

ashish.sai@maastrichtuniversity.nl

Introduction

Malware, including viruses, worms, rootkits, and botnets, exploits software vulnerabilities to propagate, evade detection, and resist removal, highlighting the need to learn from past failures to improve security design.

Defining Malware

Malware is software intentionally designed to harm users or systems, causing damage to **data, software, hardware, or privacy**. **Harmful software**, a broader class, includes unintentional damage from design or implementation errors.

Characteristics

- **Intention:** Designed to harm.
- **Damage:** Affects data, software, hardware, or privacy.
- **User Approval:** Runs without explicit user approval.

Examples

- **Viruses**
- **Worms**
- **Trojan Horses**
- **Ransomware**
- **Botnets**

How Malware Gets Onto Devices

Web-Based Attacks

- **Websites:** Links in phishing emails, search results, ads directing traffic to compromised sites.
- **Pharming Attacks:** Disrupt IP address resolution to misdirect browsers.

Downloaded Executables

- **Repackaged Software:** Legitimate software modified to include **malware**.
- **Pure Malware:** Software that is purely malicious or contains hidden malicious functionality.

Drive-by Downloads

- **Browser Vulnerabilities:** Exploiting browser vulnerabilities to install malware without **user knowledge**.

Network Worms and Email Viruses

- **Network Worms:** Exploit **vulnerabilities** in network services to spread.
- **Email Viruses:** **Malicious** attachments in emails spread malware.

Source Code Repositories and Hardware

- **Embedded Malware:** Malware in **source code repositories** or firmware.
- **Compromised Supply Chains:** Malicious firmware or **hardware**.

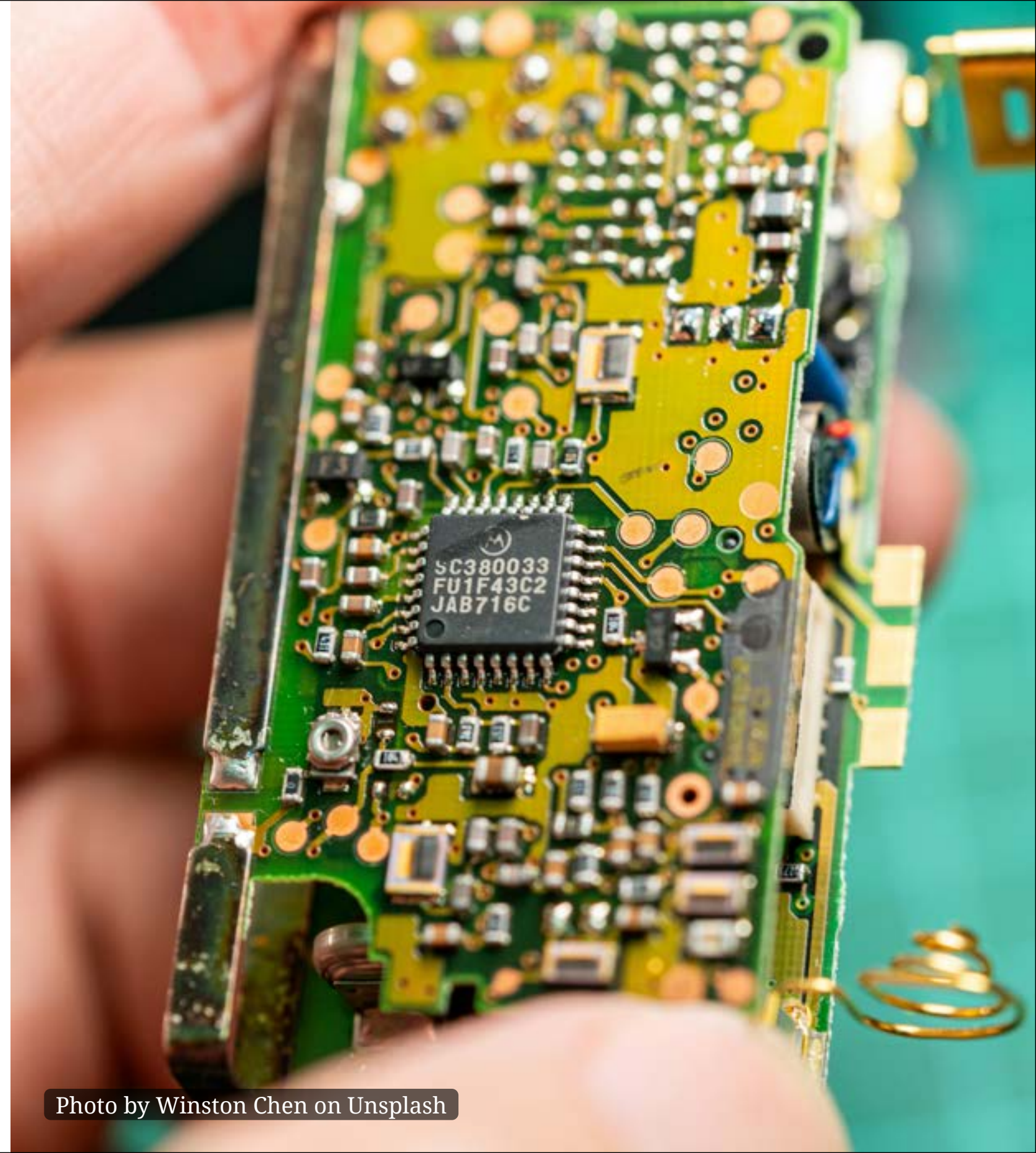


Photo by Winston Chen on Unsplash

What Makes Malware Hard to Detect

Context-Dependent

- **Legitimate Software:** Software can be malware if installed by an attacker.

Differing opinions on whether ad-displaying software is malware!

Undecidability and Anti-Detection Design

- Complete malware detection is impossible. (**Theoretical Proof**)
- **Aggressive Design:** Malware is designed to avoid detection and reverse-engineering.

Preventing Malware Installation

1. **Restricting Software Installation:** Limits user freedom, causing inconvenience and unpopularity.
2. **User Education:** Requires ongoing investment in awareness and training.
3. **Code-Signing and Anti-Malware Tools:**
 - **Code-Signing:** Verifies executable content.
 - **Anti-Malware:** Detects and removes known threats but offers partial protection.
4. **Reinstallation:** Used in severe cases to fully remove malware by reinstalling the OS and applications.

Software Churn and Malware

The ease of modern software installation and updates accelerates progress but also enables malware. Users frequently install misrepresented software with hidden functions, while high software turnover in networks and devices increases malware risks.

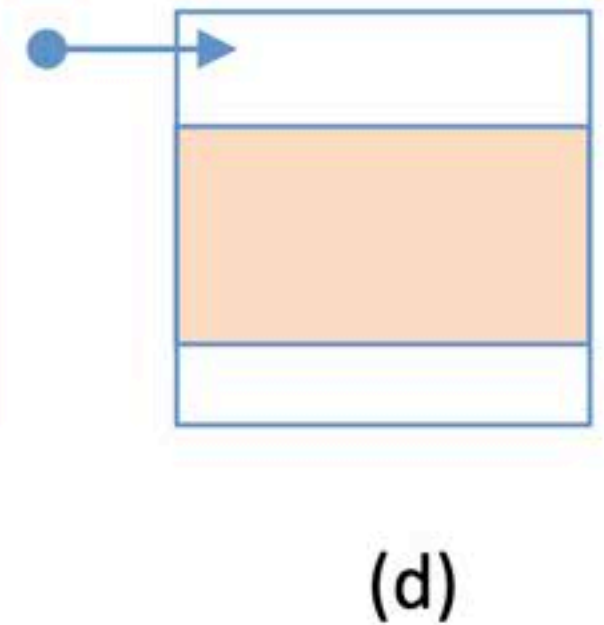
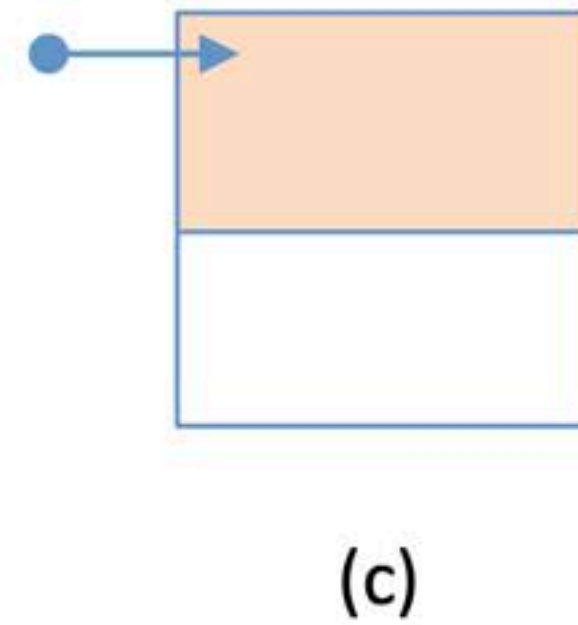
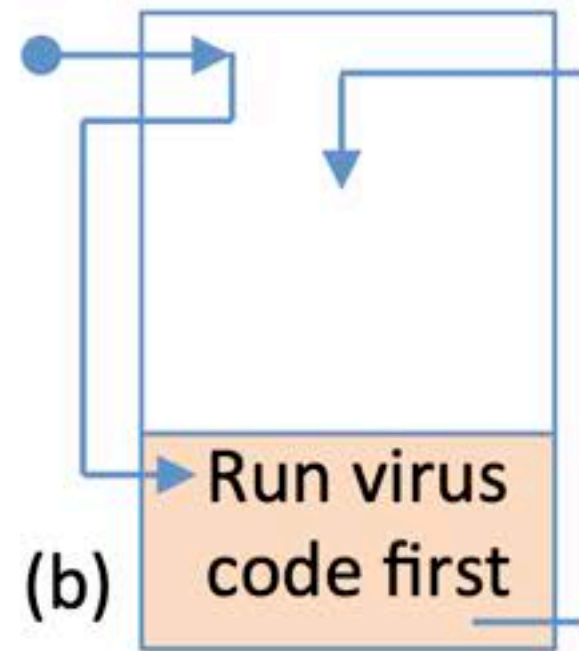
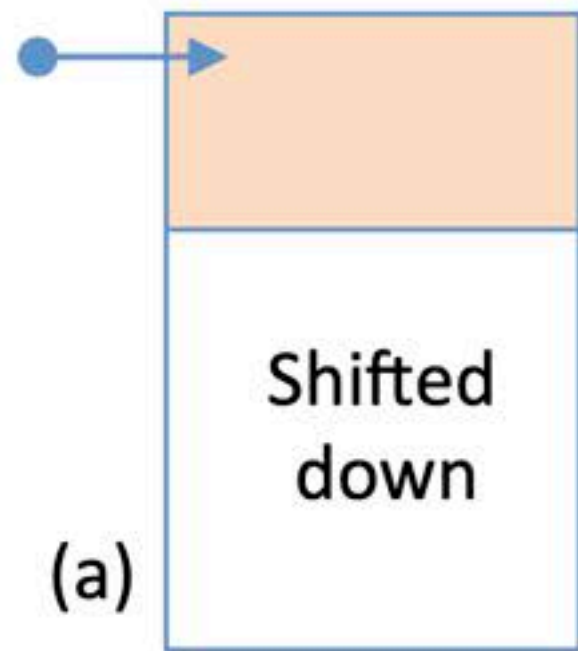
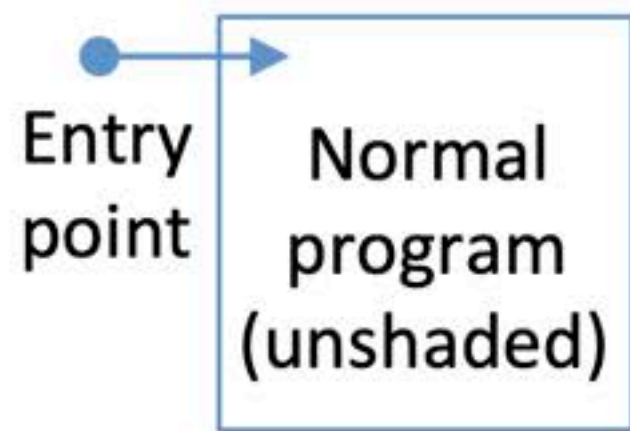
Viruses

Definition

A virus infects programs or files, **replicating** itself. It **spreads** with **human action** (e.g., USB drives, email attachments) and infects executable content.

Structure

1. **Dormancy:** Inactive until the host runs.
2. **Propagation:** Spreads to other files or machines.
3. **Trigger Condition:** Activates the payload.
4. **Payload:** Delivers functionality, from benign to severe.



Examples

- **Brain Virus (1986)** ¹: First PC virus, spread via floppy disks.
- **CIH Chernobyl Virus (1998-2000)** ²: Destructive, overwrote hard disk sectors and BIOS.

1. <https://www.historyofinformation.com/detail.php?id=1676>

2. <https://www.computerhope.com/vcih.htm>

Worms

Definition

Worms propagate automatically and continuously, leveraging network protocols and vulnerabilities. They do not require user interaction.

Differences from Viruses

1. **Automatic Propagation:** No user action needed.
2. **Network Spread:** Exploits network protocols.
3. **Vulnerability Exploitation:** Uses software vulnerabilities rather than social engineering 1.

1. **Social Engineering:** A manipulation technique used to exploit human behavior and gain unauthorized access to systems, data, or resources, often through deception or psychological tactics.

Example

- **Morris Worm (1988):**
First wide-scale worm,
exploited multiple
software
vulnerabilities.



Email-Based Malware

Combines virus and worm properties, spreading via email attachments and client features. Often involves social engineering.

Example

- **Happy99 (1999):** Convincing users to run an attached executable.

Anti-Detection Techniques

Encrypted Body

- **Simple Encryption:** **Fixed** mappings or symmetric-key encryption.

Polymorphic

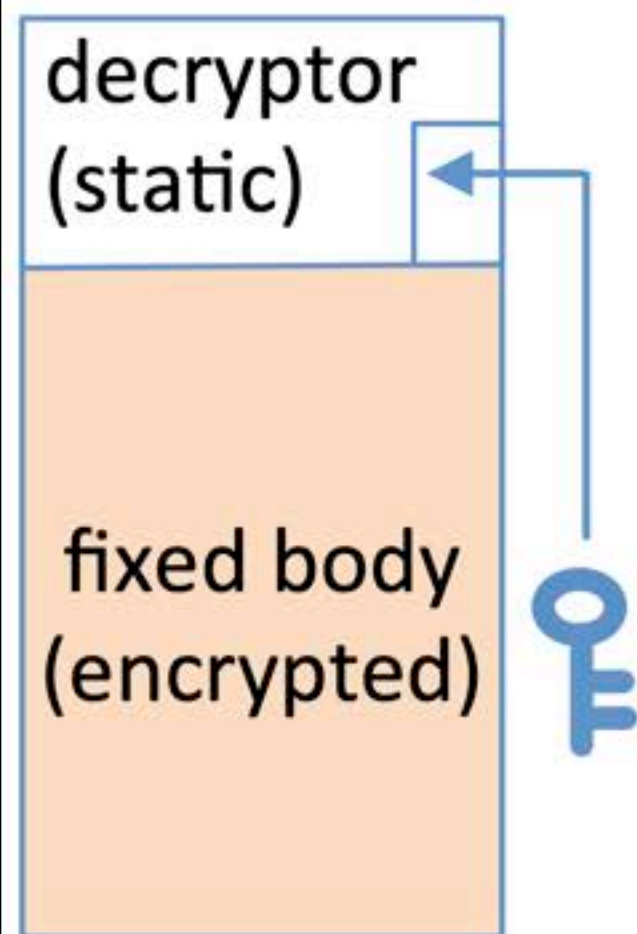
- **Mutating Decryptor:** Changes **decryptor** portions to avoid detection.

External Decryption Key

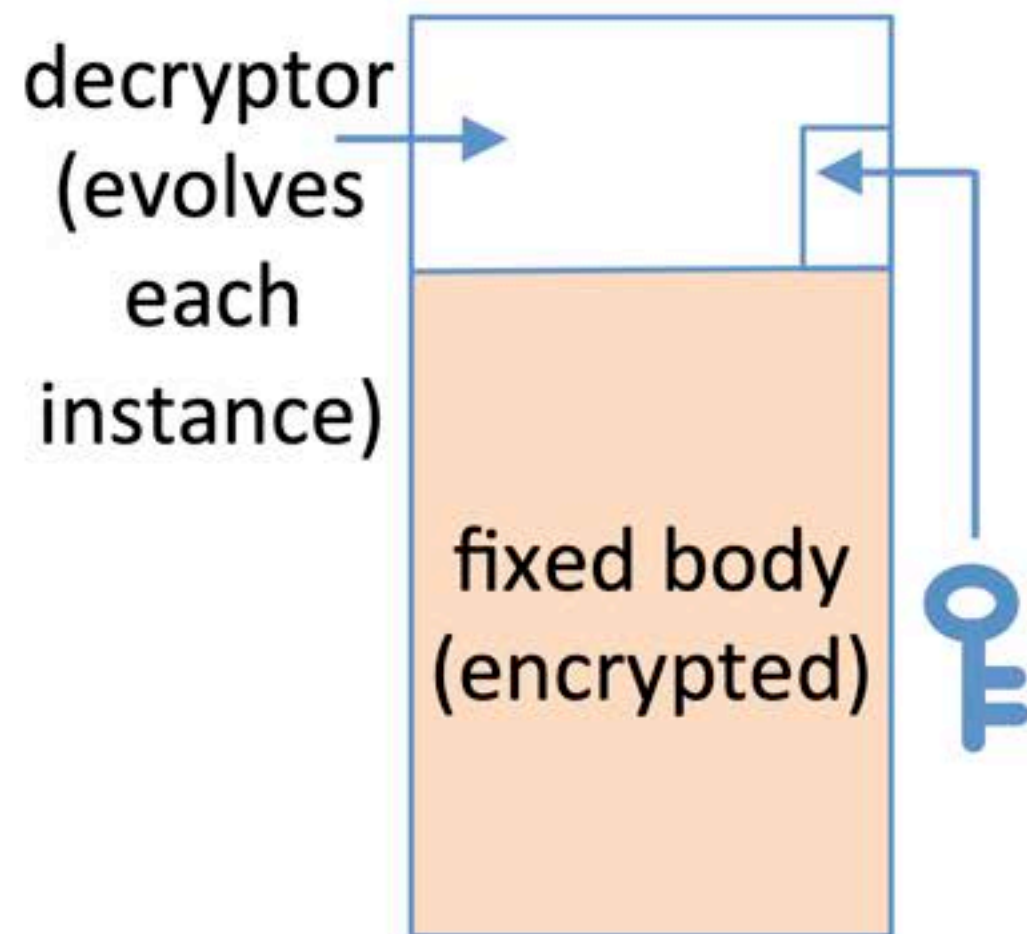
- **External Key Storage:** Key stored **outside** the infected file.

Metamorphic

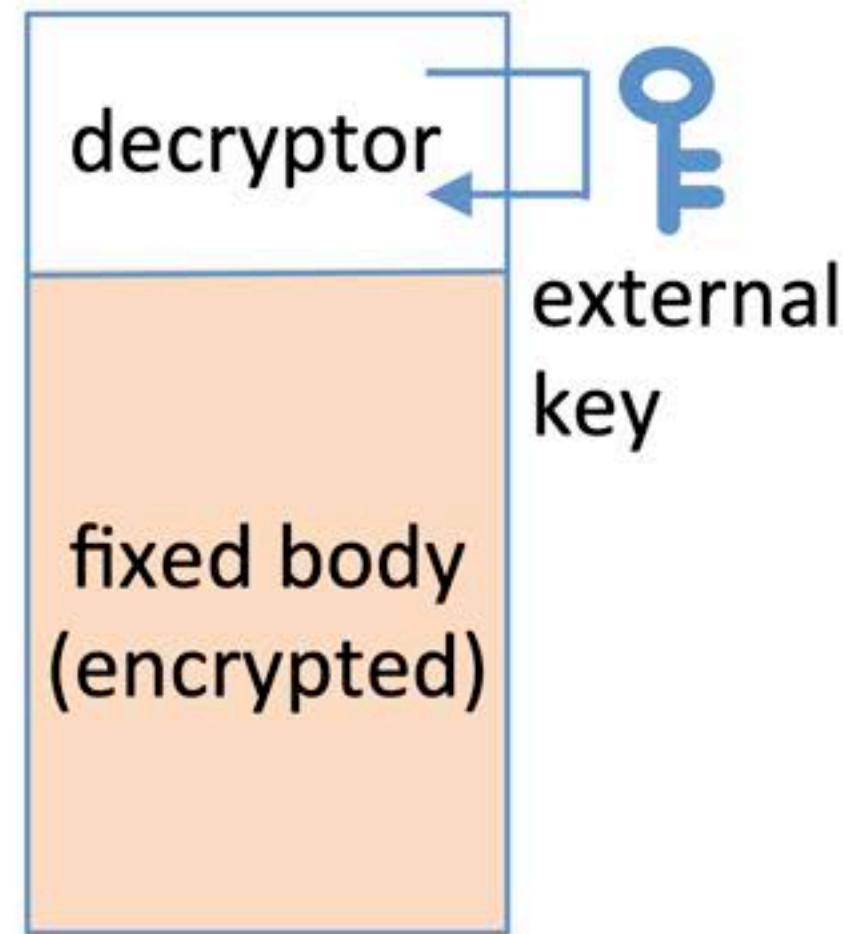
- **Rewriting Code:** Virus **rewrites** its own code on a per-infection basis.



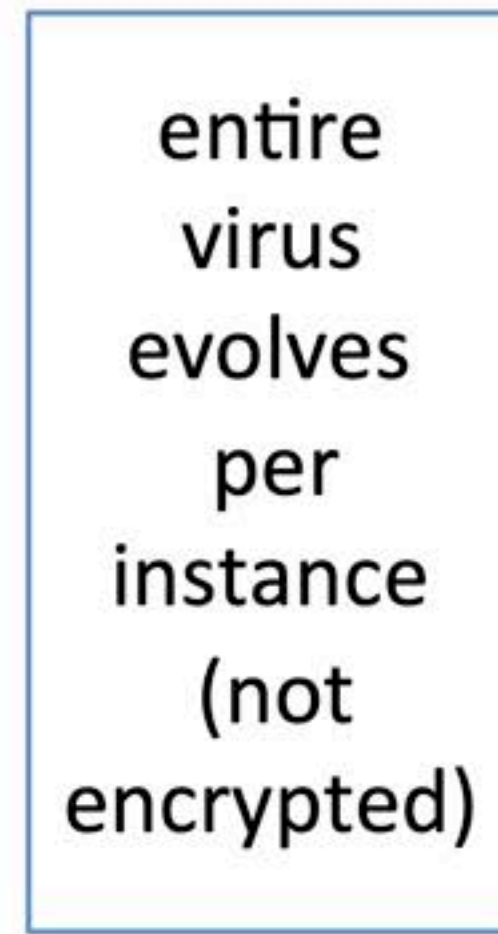
(a)



(b)



(c)



(d)

Techniques to Remain Hidden

1. **Direct Kernel Object Modification (DKOM):** Alters kernel¹ data structures.
2. **Postprocessing System Call Results:** Prunes results before returning them.

1. Kernel: The core component of an operating system, managing hardware resources and enabling communication between hardware and software.

**Stealth: Trojan
Horses, Backdoors,
Keyloggers,
Rootkits**



Photo by Tayla Kohler on Unsplash

Trojan Horse

- Software delivering malicious functionality *along with, or instead of*, its purported functionality.
- **Examples:** Fake updates (e.g., Java, Flash), free applications (e.g., screen savers).

Backdoors

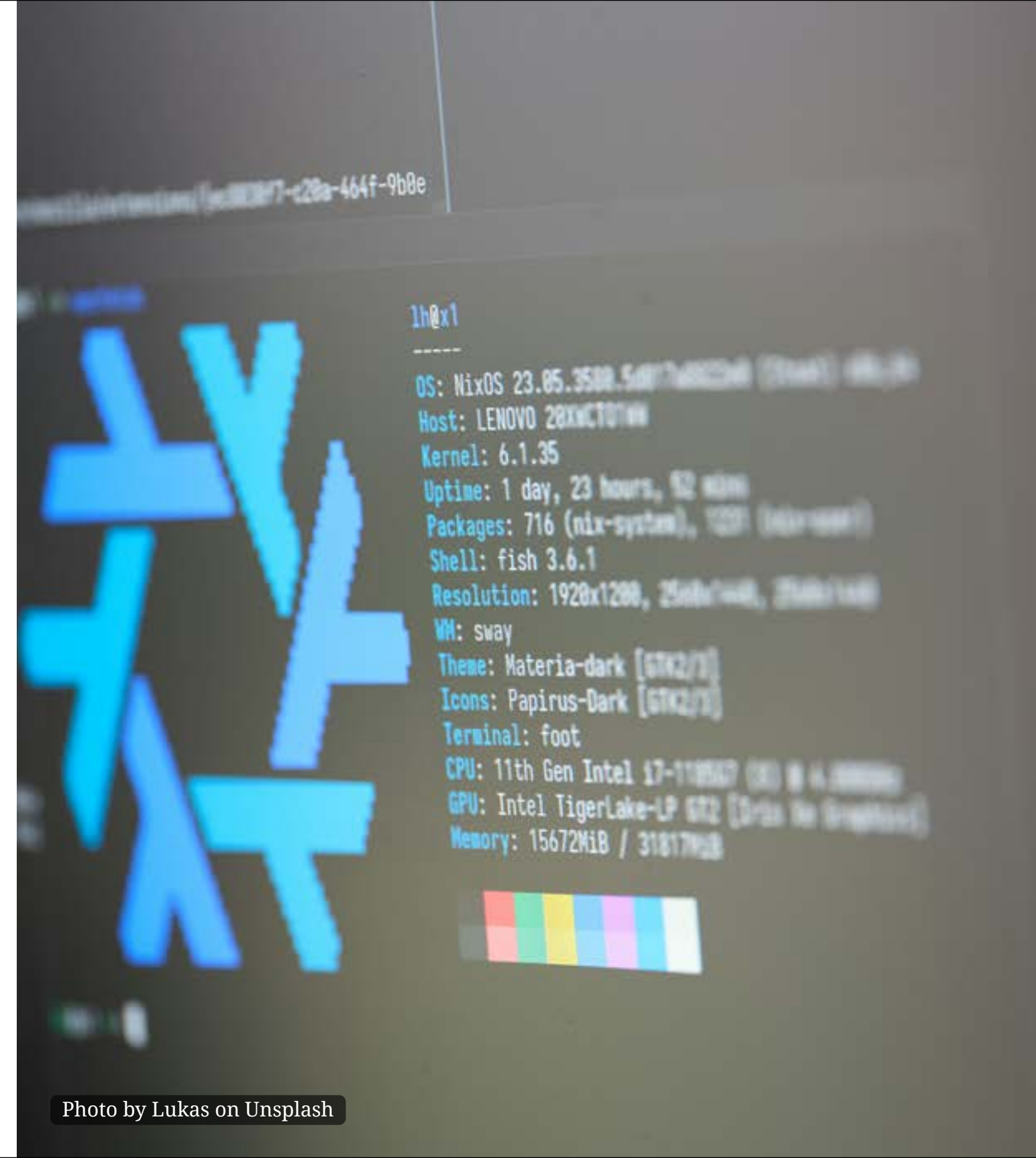
- **Definition:** Unauthorized access points bypassing normal authentication mechanisms.
- **Usage:** Often included in Trojan software and rootkits.

Keyloggers

- **Definition:** Software that records user keystrokes.
- **Targets:** Credit card details, passwords for online banking, corporate accounts, etc.

Rootkits

- **Definition:** Surreptitiously installed software components that hide their presence and facilitate malicious activities.
- **User Mode Rootkits:** Run in user space with superuser privileges.
- **Kernel Mode Rootkits:** Run in kernel space, accessing kernel resources and memory.



Example: Rootkit Goals

1. **Backdoor Functionality:** Ongoing remote access.
2. **Keyloggers:** Recording keystrokes.
3. **Surveillance:** Using device microphones, webcams, sensors.

application

shared library

user space

kernel

syscall dispatcher

syscall
table:

syscall #0

#1

#2

#i

(c) substitute
table

syscall code

substitute code

(a)

(b)

(c)

Rootkit Detail: Installation, Object Modification, Hijacking

Hijacking System Calls

- **Method 1: Hooking:** Redirects calls to rootkit code, allowing it to intercept and modify results.
- **Method 2: Overwriting:** Replaces system call code with malicious code.
- **Method 3: Substituting:** Replaces the entire syscall table with a modified one.

Windows Function Hooking

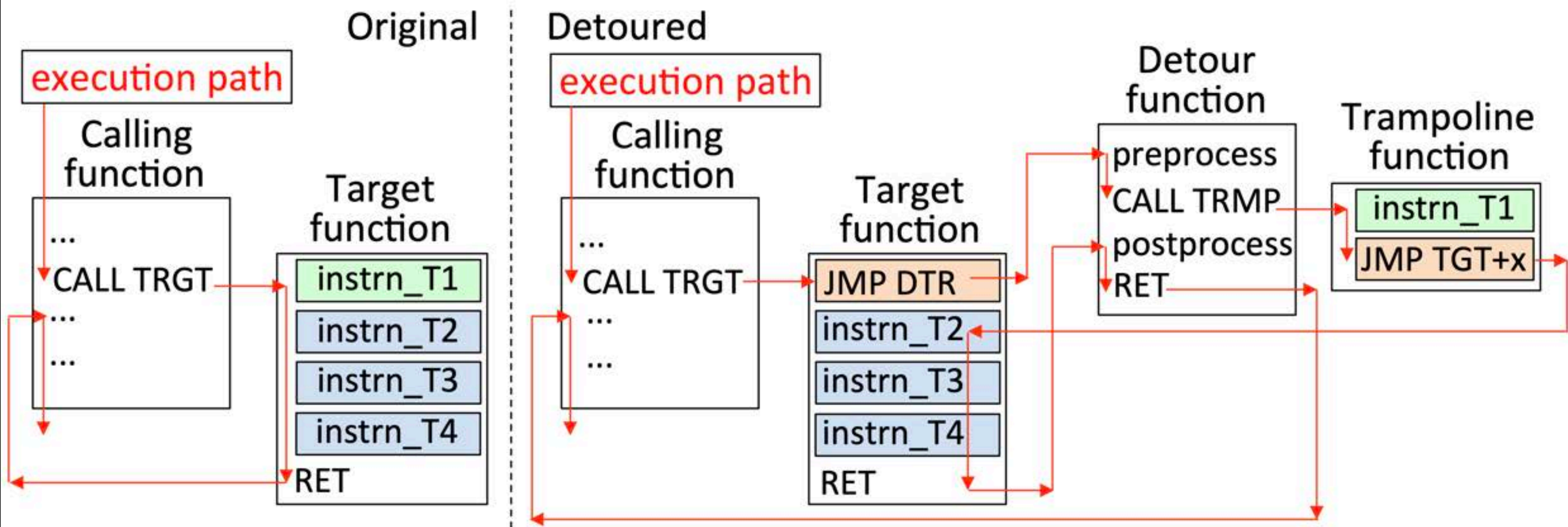
- **Kernel Space:** Hooks `SSDT` (System Service Dispatch Table).
- **User Space:** Hooks `IAT` (Import Address Table).
- **NTAPI/Native API:** Wrapper functions in `ntdll.dll` library.

Inline Hooking

- **Detour Patching:** Uses detour and trampoline functions to insert code before and after the original function.
- **Detection:** Cross-checking table addresses or hashing the target function for integrity.

Kernel Object Modification and Pruning Reports

- **DKOM:** Directly alters kernel data structures.
- **Postprocessing:** Prunes results of system calls before returning them to hide rootkit-related activities.



Installing Rootkits

1. **Kernel Module Installation:** Loading a malicious kernel module.
2. **Exploiting Vulnerabilities:** Buffer overflow in kernel code.
3. **Modifying Boot Process:** Rogue boot loader altering the kernel.
4. **Swapping Memory:** Modifying swapped kernel memory.
5. **Physical Address Access:** Using DMA writes to alter kernel memory.

Loadable Kernel Modules (LKMs)

- Executable code that can be added or removed from a running kernel to extend functionality.
- **Process:** Compiler generates object files, linker combines them into executable files, loader moves them to memory, and dynamic linker resolves shared libraries.

User Mode Rootkits

- **Operation:** Intercepts resource enumeration APIs in user processes.
- **Detection:** Cross-view difference approach compares results from different API calls to detect discrepancies.

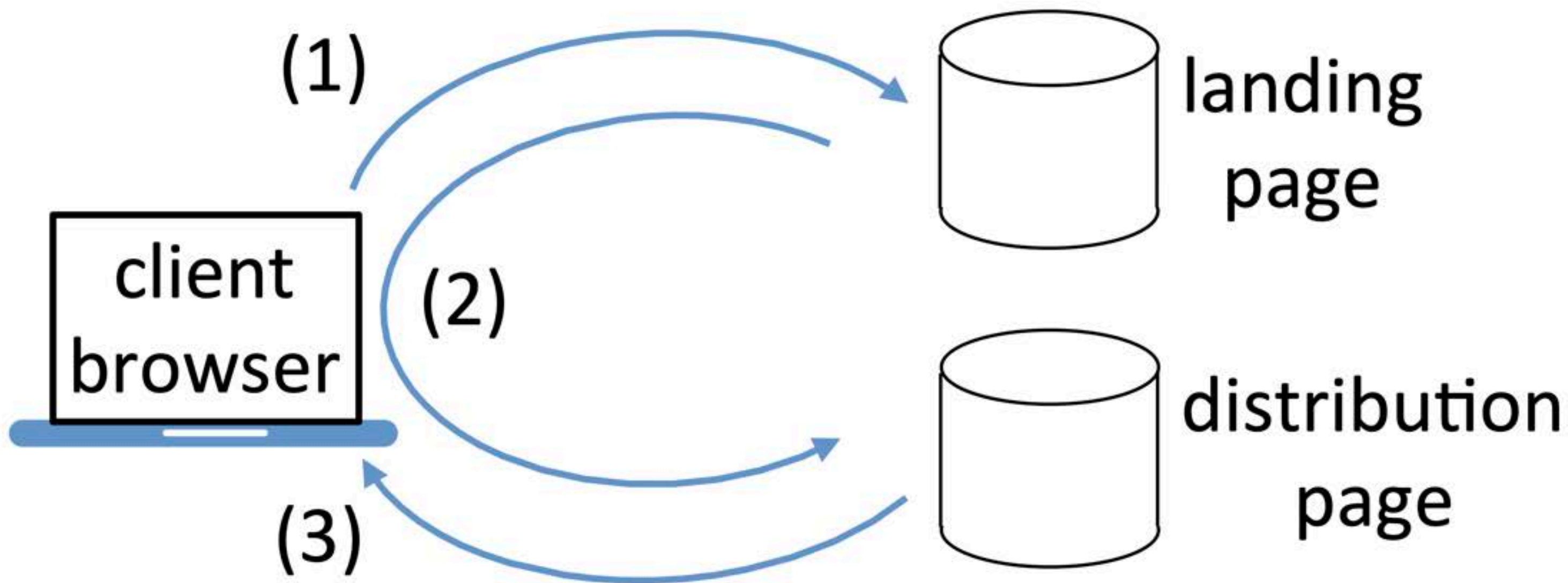
Protecting Secrets and Local Data

- **Encrypted Filesystems:** Automatically encrypt data stored to the filesystem and decrypt data upon retrieval.
- **Disk Encryption:** Software or hardware-supported encryption for all disk storage.
- **RAM Encryption:** Protects secrets such as passwords and cryptographic keys in memory.

Drive-By Downloads and Droppers

Drive-By Downloads

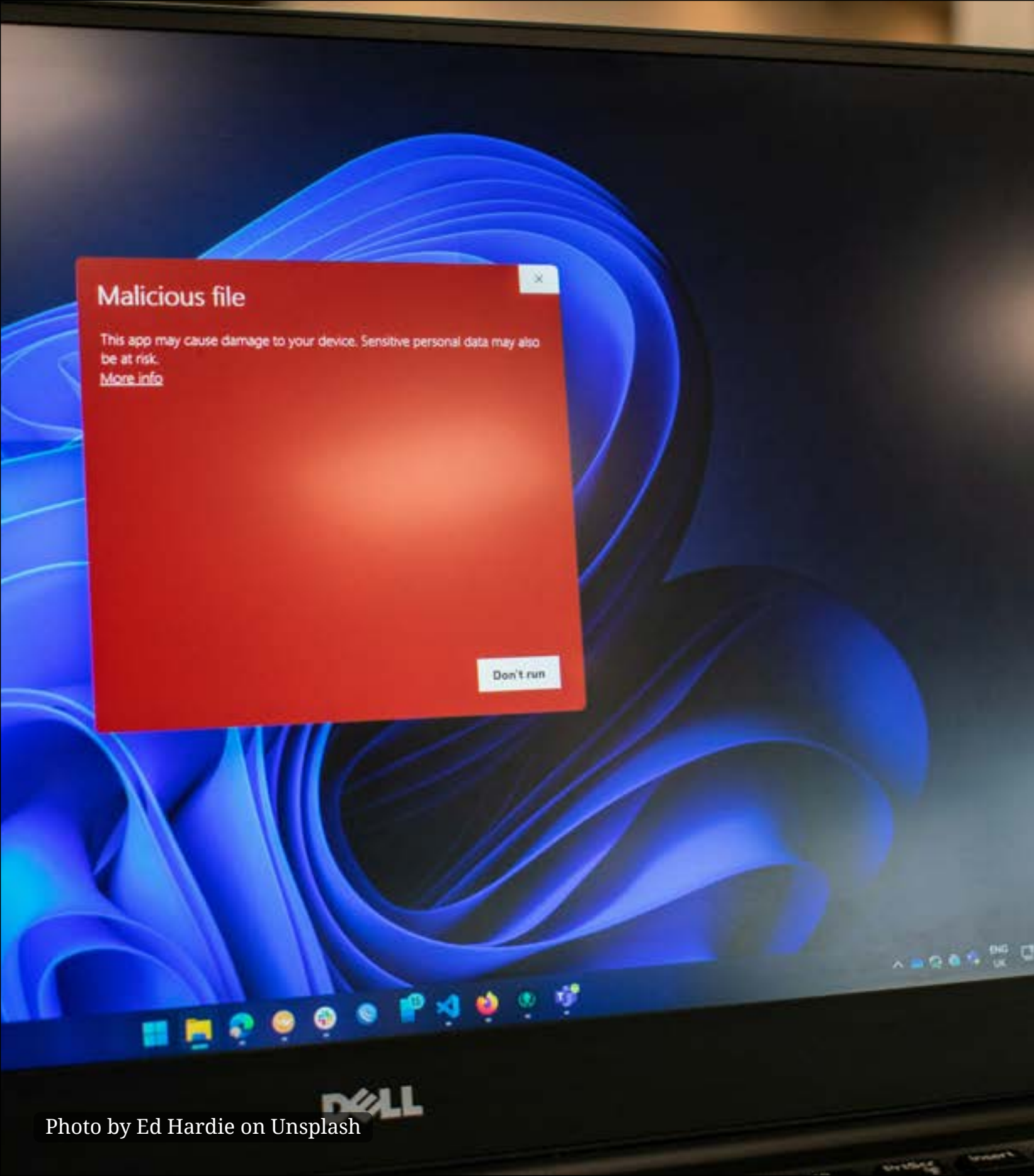
- **Exploitation:** Embeds `malicious scripts` in web pages to exploit browser vulnerabilities.
- **Redirection:** Involves multiple redirect hops to download binaries.
- **Deployment:** Spreads `keyloggers`, `backdoors`, `rootkits`, and `botnet` recruitment.



Droppers and Downloaders

- Droppers install other **malware**, and downloaders specifically download additional components.
- **Example: Babylonia (1999):** An early dropper that downloaded and executed additional files.

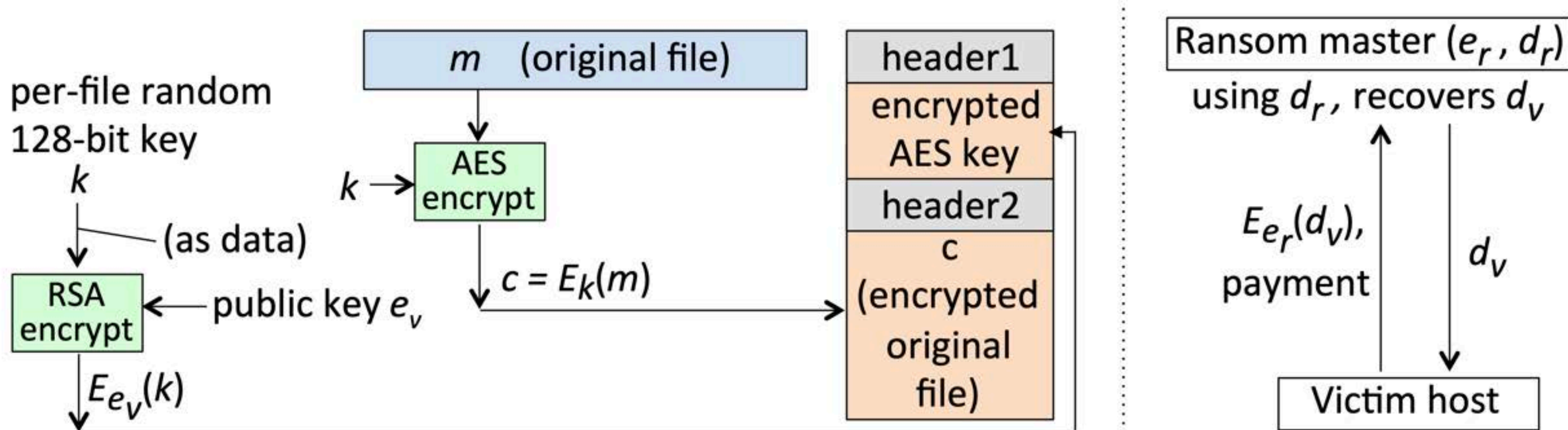
Ransomware, Botnets, and Other Beasts



Ransomware

- **Definition:** Encrypts user files and demands payment for decryption keys.
- **Asymmetric File Locking:** Uses public-key cryptography to encrypt symmetric keys.
- **Example: WannaCry (2017):** Infected over 200,000 computers, demanding Bitcoin payments.

Ransomware master public key e_r used to encrypt d_v to $E_{e_r}(d_v)$
Per-victim (public, private) key pair (e_v, d_v) generated on victim host



Botnets and Zombies

- Compromised computers (bots) controlled remotely, forming a network (botnet).
- **Communication Structures:**
 1. **Client-Server Model:** Central administrative server.
 2. **Peer-to-Peer:** Coordination over network protocols.
 3. **Multi-Tiered Hierarchies:** Insulating bot herders from zombies.

Other Beasts

- **Zero-Day Exploits:** Attacks using unknown vulnerabilities.
- **Logic Bombs:** Malicious code triggered by specific conditions.
- **Rabbits:** Rapidly replicating malware that consumes resources.
- **Easter Eggs:** Harmless hidden features in software.

Social Engineering

Social Engineering

- Tricking users into downloading, installing, and executing malware through deceptive tactics.
- **Example: Happy99 (1999):** An email worm-virus convincing users to run an attached executable.

Categorizing Malware

- **Based on Propagation:** Viruses, worms, drive-by downloads.
- **Based on Payload:** Ransomware, keyloggers, backdoors.
- **Based on Stealth:** Rootkits, logic bombs.

Category name	Property	**** (blank denotes: no)	****	****
	BREEDS†	HOSTED	STEALTHY	VECTOR
virus	3	3		U
worm	3			N
Trojan horse		3	3	E or S
backdoor		maybe	3	T or S
rootkit, keylogger			3	T or S
ransomware				T
drive-by download	H		3	S